Class 17: Optimizing Spark Applications (The Catalyst Optimizer)

DSL351 / DSP352: Big Data Analytics

Amit Kumar Dhar

ED1, 406A, amitkdhar@iitbhilai

March 23, 2026

Introduction to Spark Optimization

The Need for Optimization

- Big Data processing involves massive datasets and complex transformations.
- Writing efficient code manually is difficult and error-prone.
- Spark abstracts the execution details, but relies on an engine to make the execution fast.
- Without optimization, even simple queries might result in unnecessary data shuffling or full table scans.

RDDs vs. Structured APIs

- **RDDs:** Spark does not understand the inner structure of the data or the semantics of the transformations. It just sees a sequence of opaque functions (e.g., in Scala/Python).
- **DataFrames/Datasets:** Spark understands the schema (data types, column names) and the exact operations (filter, select, groupBy).
- This semantic understanding allows Spark to automatically optimize the execution.

What is the Catalyst Optimizer?

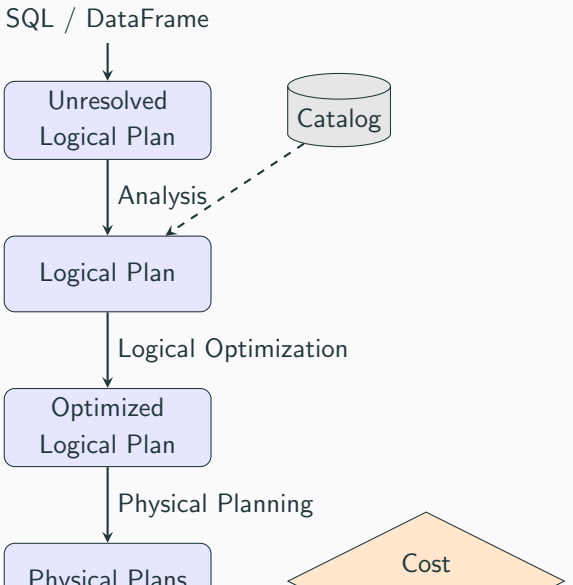
- Catalyst is Spark SQL's query optimizer framework.
- It is written in Scala and leverages Scala's functional programming features (pattern matching).
- It transforms a user's SQL query or DataFrame DSL code into an optimized physical execution plan.
- It continuously evolves with each Spark release, adding new optimization rules.

Core Concepts of Catalyst

- **Trees:** Catalyst represents queries as trees (AST - Abstract Syntax Trees). Nodes are operators (e.g., Filter, Project) or expressions.
- **Rules:** Catalyst applies a set of rules to transform one tree into another (e.g., moving a Filter before a Join).
- **Pattern Matching:** Used extensively to find sub-trees that match specific structures and apply rules to them.

The Catalyst Optimization Pipeline

Catalyst Pipeline Overview



Phase 1: Analysis

- Starts with an **Unresolved Logical Plan**.
- "Unresolved" means the query is parsed, but the column names or table names haven't been validated against actual data.
- Example: `SELECT age FROM users` – Does the `users` table exist? Does it have an age column?
- Catalyst uses the **Catalog** (a repository of all table and DataFrame metadata) to resolve these references.
- Output: **Resolved Logical Plan**.

Phase 2: Logical Optimization

- Applies rule-based optimizations (heuristics) to the Logical Plan.
- The goal is to simplify the query without changing its result.
- Hundreds of rules are applied in batches until the plan converges (stops changing).
- Output: **Optimized Logical Plan**.

Phase 3: Physical Planning

- Spark takes the Optimized Logical Plan and generates one or more **Physical Plans**.
- A Physical Plan details exactly *how* the operations will execute on the cluster (e.g., using a BroadcastHashJoin vs. SortMergeJoin).
- If multiple plans are generated, a **Cost Model** (Cost-Based Optimizer - CBO) estimates the cost of each and selects the cheapest one.
- Output: **Selected Physical Plan**.

Phase 4: Code Generation

- Translates the Selected Physical Plan into optimized Java/Scala bytecode.
- This heavily relies on **Project Tungsten** (Whole-Stage Code Generation).
- By compiling parts of the query into a single function, it removes the overhead of virtual function calls.
- Output: **Executable RDD DAG**.

Deep Dive into Logical Optimization Rules

Predicate Pushdown

- Pushes `FILTER` operations as close to the data source as possible.
- Reduces the amount of data read from disk and transferred across the network.

Unoptimized:

1. Read entire Users table.
2. Filter where $age > 30$.

Optimized (Pushdown):

1. Ask the data source (e.g., Parquet) to only return rows where $age > 30$.

Predicate Pushdown Example

Scala:

```scala
// Spark will optimize this by pushing the filter to the data source
val df = spark.read.parquet("users.parquet")
val adults = df.filter($"age" > 18).select("name")

```

Python:

```python
# PySpark equivalent
df = spark.read.parquet("users.parquet")
adults = df.filter(df.age > 18).select("name")

```

Column Pruning

- Only reads the columns that are strictly necessary for the final output.
- Highly effective with columnar formats like Parquet and ORC.

Example: If a table has 100 columns, but the query is `SELECT name FROM users WHERE age > 18`, Spark will only read the `name` and age columns. The other 98 columns are never loaded into memory.

Constant Folding

- Evaluates constant expressions at compile time rather than runtime.
- Avoids redundant calculations for every row.

Unoptimized: `SELECT price + (10 * 0.1) FROM sales`
(calculates 10×0.1 for every row)

Optimized: `SELECT price + 1.0 FROM sales`
(calculates once during plan generation)

Constant Folding Example

Scala:

```scala
import org.apache.spark.sql.functions.lit

val df = spark.range(100)
// The expression (lit(5) * lit(10)) is evaluated once by Catalyst
val result = df.select($"id", (lit(5) * lit(10)).as("constant_val"))

```

Python:

```python
from pyspark.sql.functions import lit

df = spark.range(100)
result = df.select(df.id, (lit(5) * lit(10)).alias("constant_val"))

```

Join Reordering

- When multiple tables are joined, the order of joins drastically affects performance.
- Catalyst uses statistics (if available) to estimate table sizes.
- It reorders joins to join the smallest tables first, reducing the intermediate data size.

Project Tungsten & Code Generation

What is Project Tungsten?

- A major initiative introduced in Spark 1.5 to improve CPU and memory efficiency.
- Focus shifted from optimizing I/O (which was largely solved) to optimizing CPU and memory bottlenecks.
- Three main pillars:
  1. Memory Management and Binary Processing
  2. Cache-aware Computation
  3. Whole-Stage Code Generation

Memory Management and Binary Processing

- **Problem:** Java objects have a large memory overhead (e.g., a simple 4-byte string might take 48 bytes in the JVM) and cause Garbage Collection (GC) pauses.
- **Tungsten Solution:** Spark stores data in a custom binary format directly in off-heap memory.
- Operations are performed directly on this serialized binary data without deserializing it into Java objects.
- Significantly reduces GC pressure and memory footprint.

Cache-aware Computation

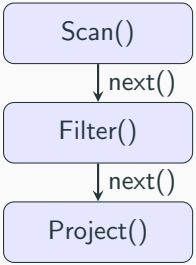
- Processors are much faster than main memory (RAM).
- CPU caches (L1/L2/L3) are used to speed up access.
- Tungsten designs algorithms and data structures (like Cache-friendly hash tables) that exploit CPU cache hierarchy.
- Result: Better CPU utilization and faster processing.

Whole-Stage Code Generation

- **Volcano Iterator Model:** Traditionally, query engines use this model where each operator has a `next()` method. This causes millions of virtual function calls, slowing down execution.
- **Tungsten Solution:** Catalyst compiles an entire query (or parts of it) into a single, highly optimized Java function at runtime.
- It eliminates virtual function calls and leverages modern compiler optimizations (like loop unrolling).

Illustration of Whole-Stage Code Generation

Without CodeGen



With Whole-Stage CodeGen

```
for (row in scan) {
  if (filter(row)) {
    project(row)
  }
}
```

Reading explain() Plans

The `explain()` Method

- To see how Catalyst optimized your query, use the `explain()` method on a DataFrame.
- It prints the execution plan to the console.
- Essential tool for debugging performance issues and verifying if optimizations (like Pushdown) occurred.

Basic `explain()`

```scala
val df = spark.read.parquet("data.parquet").filter($"age" > 25)
df.explain() // Shows only the Physical Plan
```

Output Snippet:

```
== Physical Plan ==
*(1) Project [id#0, name#1, age#2]
+- *(1) Filter (isnotnull(age#2) AND (age#2 > 25))
   +- *(1) ColumnarToRow
      +- FileScan parquet [id#0,name#1,age#2] Batched: true, DataFilters: [isnotnull(age
     #2), (age#2 > 25)], Format: Parquet, ... PushedFilters: [IsNotNull(age), GreaterThan
     (age,25)], ReadSchema: struct<id:int,name:string,age:int>
```

Deciphering the Physical Plan

- The plan reads from bottom to top.
- `FileScan parquet`: Scanning the Parquet file.
- `PushedFilters`: Confirms that Predicate Pushdown worked! Spark asked Parquet to filter `age > 25`.
- `*(1)`: Indicates Whole-Stage Code Generation is active. All nodes with `*(1)` are compiled into a single function.

Extended `explain(true)`

```
df.explain(True) # Or df.explain(mode="extended") in newer versions

```

Shows all phases of Catalyst:

1. **Parsed Logical Plan:** Raw AST.
2. **Analyzed Logical Plan:** Resolved against Catalog.
3. **Optimized Logical Plan:** After rule application.
4. **Physical Plan:** The final chosen execution strategy.

Code Examples: Comparing Plans

Example 1: Unoptimized vs. Optimized Logic

Task: Filter adults and get their names.

Scala Code:

```scala
// Bad approach (Filter after Select)
val badDF = spark.read.parquet("data").select("name", "age").filter($"age" > 18)

// Good approach (Filter before Select)
val goodDF = spark.read.parquet("data").filter($"age" > 18).select("name")

```

Result: Catalyst recognizes these are semantically identical. It will optimize `badDF` to behave exactly like `goodDF`. Both will yield the **same** Physical Plan!

Example 2: Analyzing a Join (Python)

```python
emp = spark.read.parquet("emp.parquet")
dept = spark.read.parquet("dept.parquet")

# Small table join
joined = emp.join(dept, "dept_id")
joined.explain()

```

What to look for in the plan:

- `BroadcastHashJoin`: If `dept` is small, Spark automatically broadcasts it to all executors, avoiding a shuffle.
- `SortMergeJoin`: If both are large, Spark will shuffle and sort data before joining.

Example 3: Forcing a Broadcast Join

Sometimes statistics are wrong, and Spark chooses `SortMergeJoin` when `BroadcastHashJoin` is better. You can give Catalyst a hint.

Scala:

```scala
import org.apache.spark.sql.functions.broadcast
val joined = emp.join(broadcast(dept), Seq("dept_id"))

```

Python:

```python
from pyspark.sql.functions import broadcast
joined = emp.join(broadcast(dept), "dept_id")

```

The `explain()` plan will now strictly show a `BroadcastHashJoin`.

Cost-Based Optimizer (CBO)

Cost-Based Optimization (CBO)

- While rule-based optimization (RBO) uses heuristics, CBO uses statistics about the data (table size, row count, column distributions).
- Disabled by default in older versions, often enabled in newer Spark SQL engines.
- Requires running `ANALYZE TABLE` to gather statistics.

What does CBO do?

- **Join Selection:** Accurately decides between Broadcast Hash Join vs. Sort Merge Join.
- **Join Reordering:** Finds the optimal order to join multiple tables to minimize intermediate dataset sizes.
- **Filter Estimation:** Estimates how many rows will pass a filter.

Enabling and Using CBO

SQL to compute statistics:

```
ANALYZE TABLE employees COMPUTE STATISTICS;
ANALYZE TABLE employees COMPUTE STATISTICS FOR COLUMNS age, salary;

```

Spark Configuration:

```
spark.conf.set("spark.sql.cbo.enabled", "true")
spark.conf.set("spark.sql.cbo.joinReorder.enabled", "true")

```

Adaptive Query Execution (AQE)

What is AQE?

- Introduced as a major feature in Spark 3.0.
- Standard Catalyst optimization happens **before** the query starts (static).
- AQE optimizes the plan **dynamically during runtime** based on actual runtime statistics.

Key Features of AQE

1. **Dynamically coalescing shuffle partitions:** Adjusts the number of partitions after a shuffle to prevent small, inefficient tasks.
2. **Dynamically switching join strategies:** If a table turns out to be smaller than expected after filtering, it switches from SortMergeJoin to BroadcastHashJoin on the fly.
3. **Dynamically optimizing skew joins:** Detects skewed data partitions and splits them into smaller tasks automatically.

Enabling AQE

In Spark 3.x, AQE is usually enabled by default.

Scala/Python Configuration:

```
# In PySpark
spark.conf.set("spark.sql.adaptive.enabled", "true")
spark.conf.set("spark.sql.adaptive.coalescePartitions.enabled", "true")
spark.conf.set("spark.sql.adaptive.skewJoin.enabled", "true")

```

Common Optimization Scenarios

Scenario 1: UDFs (User Defined Functions)

- **Problem:** Catalyst cannot see inside UDFs (they are opaque boxes).
- Code inside a UDF cannot be optimized, and Predicate Pushdown cannot pass through a UDF.
- **Solution:** Always try to use built-in Spark SQL functions instead of UDFs. Built-in functions are fully optimized by Catalyst and Tungsten.

UDF vs Built-in Example

Bad (Python UDF):

```
from pyspark.sql.functions import udf
from pyspark.sql.types import IntegerType

@udf(returnType=IntegerType())
def add_one(val):
    return val + 1

df.withColumn("new_age", add_one(df.age)) # Opaque to Catalyst

```

Good (Built-in):

```
df.withColumn("new_age", df.age + 1) # Fully optimized by Catalyst

```

Scenario 2: Data Skew

- **Problem:** One partition has significantly more data than others, causing one task to run much longer (the straggler).
- **Symptom:** In Spark UI, 199 tasks finish in seconds, but 1 task takes 10 minutes.
- **Solution 1:** Enable AQE Skew Join Optimization.
- **Solution 2:** Salting. Add a random key to the skewed column to distribute the data evenly, perform the operation, and then aggregate again.

Scenario 3: The Small File Problem

- **Problem:** Reading thousands of tiny files creates massive overhead for the NameNode and results in too many small partitions in Spark.
- **Solution:** Use `coalesce()` before writing data to reduce the number of output files.
- Or, compact files periodically using a separate Spark job.

Questions?